

PROJECT REPORT

ON

"GUI-Based Quiz Application in Java"

Submitted by:

[Student Name]

Roll Number: XXXXXXXX

Branch: Computer Science & Engineering

Year: Final Year Diploma

Under the guidance of:

[Mentor Name]

[Institute Logo]

Lecturer, Department of Computer Science & Engineering

Government Polytechnic, [City Name]

(2024–2025)

**Your
Certificate
Copy**

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to all those who have helped me directly or indirectly in the successful completion of my project "**Tic Tac Toe Game in Java**".

First and foremost, I would like to thank **[Guide Name]**, my project guide, for their constant support, guidance, and valuable suggestions throughout the project development.

I am also grateful to **[HOD Name]**, Head of the Computer Science Department, for providing the necessary facilities and a conducive environment for carrying out this work.

I would also like to thank all the faculty members and my classmates who encouraged and motivated me during the project work.

Finally, I would like to thank my parents and family for their continuous support and blessings.

(Signature)

[Institute Name]

Abstract

This project report presents the design and implementation of a **Graphical User Interface (GUI)-based Tic Tac Toe game** using the Java programming language. Tic Tac Toe, also known as "Noughts and Crosses," is a classic two-player strategy game played on a 3x3 grid. The primary goal of this project is to replicate the manual paper-and-pencil version of the game into a computerized form with added interactivity, efficiency, and user-friendliness.

The project has been developed using **Java Swing**, which provides the graphical components necessary to create the game interface. The players can take turns by clicking on buttons that represent the grid cells, and the program automatically checks the winning condition after every move. If a player succeeds in placing three identical marks in a row, column, or diagonal, the program declares the winner. In case the grid is completely filled without a winner, the game ends in a draw.

The main focus of this project is on **event-driven programming** in Java, where user actions such as button clicks are captured and processed to update the game state. The project also demonstrates **object-oriented programming principles** like encapsulation and modularity, which make the code easy to maintain and extend.

The Tic Tac Toe project not only provides entertainment but also strengthens programming knowledge by combining logic, design, and graphical user interface development. This project is useful for beginners in computer science to understand how theoretical concepts of Java can be applied to create real-world applications.

Introduction

Tic Tac Toe is one of the most popular and simplest games played across the world. It is generally played by two players on a 3x3 grid where each player takes turns to mark either **X** or **O** in one of the available empty cells. The player who successfully places three of their marks in a row, column, or diagonal wins the game. If all the cells are filled without any winning condition being satisfied, the game ends in a draw.

The game is also known as "Noughts and Crosses" in some countries. Although the game is very simple, it has a strong role in building logical thinking and strategic decision-making skills. Since the number of possible moves is limited, it is a good starting point for programmers to learn problem-solving.

In the modern computing era, Tic Tac Toe is often used as a beginner project in various programming languages such as C, C++, Java, and Python. Developing the game on a computer system allows students to understand how to convert manual rules into logical code. This project uses **Java Swing** to create a user interface, where two players can play interactively using a computer. The implementation focuses on event-driven programming, graphical components, and winner-checking logic.

Objectives:

The main objective of this project is to design and implement a GUI-based Tic Tac Toe game using Java programming language. The specific objectives include:

1. To provide an interactive platform where two players can play Tic Tac Toe on a computer.
2. To understand and apply the concept of **Java Swing components** such as `JFrame`, `JButton`, and `JLabel`.
3. To implement **event-driven programming** by handling button clicks through `ActionListener`.
4. To design the logic for determining the winner by checking rows, columns, and diagonals.
5. To provide additional functionality such as a restart option so that players can play multiple rounds.
6. To improve programming and problem-solving skills by working on a real-world mini project.

By achieving these objectives, students will gain practical knowledge of GUI programming and logical thinking in software development.

System Requirements:

Hardware Requirements:

To run the Tic Tac Toe project, the following minimum hardware configuration is required:

- **Processor:** Intel i3 or higher (Dual Core minimum)
- **RAM:** 4 GB or more
- **Hard Disk:** At least 1 GB free space
- **Monitor:** Standard resolution (1024x768 or higher)

Software Requirements:

- **Programming Language:** Java (JDK 8 or above)
- **Integrated Development Environment (IDE):** IntelliJ IDEA / Eclipse / NetBeans
- **Operating System:** Windows 7/8/10/11, Linux, or macOS
- **Additional Libraries:** Java Swing (already part of JDK)

This ensures that the project runs smoothly on any normal computer system used by students.

Tools and Technologies Used:

In this project, the following tools and technologies have been utilized:

1. **Java Programming Language:** A platform-independent, object-oriented language that provides a strong foundation for software development.
2. **Java Swing:** A part of Java Foundation Classes (JFC) used for building graphical user interfaces. Swing provides components like buttons, labels, text fields, and frames that are used in this game.
3. **AWT Event Handling:** Abstract Window Toolkit (AWT) concepts are used for handling user actions such as button clicks.
4. **Object-Oriented Programming (OOP):** Concepts like classes, objects, and methods are applied in the code to organize logic.
5. **IDE (IntelliJ IDEA):** Used to write, debug, and run the Java program efficiently.

Using these tools, a simple yet powerful interactive game is created.

Problem Definition:

Traditionally, Tic Tac Toe is played on paper where players draw a 3x3 grid and mark their turns with a pen or pencil. While this method is simple, it has certain limitations such as lack of interactivity, repeated drawing of grids, and difficulty in tracking multiple rounds.

The computerized version solves these problems by providing:

- An automatically generated grid on the screen.
- Instant winner detection without manual checking.
- Easy restart option for playing multiple matches.
- A user-friendly interface where mistakes such as overwriting a move are prevented.

Hence, the problem is defined as:

"To design a computerized version of the Tic Tac Toe game using Java Swing that allows two players to play interactively, automatically detects winners/draws, and provides a restart option."

System Design:

System design is one of the most important phases of software development because it transforms the requirements and problem definition into a structured solution. In the Tic Tac Toe project, the system design focuses on how the game components interact with each other, how the flow of the program works, and how the user interface communicates with the game logic.

Design Approach

The project has been designed using a **modular approach** where each task is divided into smaller, manageable parts. The major modules are:

1. **User Interface Module** – Responsible for displaying the 3x3 game board using buttons and handling user input through clicks.
2. **Game Logic Module** – Contains the logic to check winning conditions, draw conditions, and switching turns between players.
3. **Control Module** – Connects the user interface and game logic, ensuring smooth communication between them.

Flow of the Game

1. The game board is initialized with empty cells.
2. The first player (X) starts the game by clicking on any cell.

3. The program marks the cell and switches the turn to the other player (O).
4. After each move, the program checks for winning or draw conditions.
5. If a player wins, a message is displayed, and the game can be restarted.
6. If all cells are filled without a winner, the game declares a draw.

Sample Flowchart Steps:

- Start → Initialize Game → Player Move → Update Board → Check Win/Draw → If Win → Display Winner → End / Restart.

Implementation

Implementation is the process of converting the design into an actual working system by writing and integrating the source code. In the Tic Tac Toe project, the implementation has been done using **Python** and the **Tkinter library** for GUI development.

The implementation can be divided into different modules:

1. Importing Libraries

At the beginning of the program, we import the Tkinter library to build the graphical user interface.

```
from tkinter import *  
from tkinter import messagebox
```

- Tkinter provides the functions to create windows, buttons, and layouts.
- messagebox is used to display pop-up messages when a player wins or the game is drawn.

2. Initializing the Main Window

```
root = Tk()  
root.title("Tic Tac Toe")
```

- Tk() creates the main application window.
- title() sets the window title as *Tic Tac Toe*.
This is the container where the game board and buttons will appear.

3. Game Variables

We use some variables to keep track of the game state:

- **Player turn** – A variable to check whether it's X's turn or O's turn.

- **Button grid** – A 3x3 arrangement of buttons representing the game board.

Example:

```
turn = True    # True means X's turn, False means O's turn
```

4. Creating the Game Board

We create a 3x3 grid of buttons using nested loops:

```
for i in range(3):
    for j in range(3):
        button = Button(root, text=" ", font=('normal',40), width=5,
height=2,
                                command=lambda row=i, col=j: click(row,
col))
        button.grid(row=i, column=j)
```

- Each button is initialized as empty.
- When clicked, the button calls the `click()` function with its row and column number.

5. Handling Button Clicks

```
def click(row, col):
    global turn
    if buttons[row][col]['text'] == " ":
        if turn:
            buttons[row][col]['text'] = "X"
        else:
            buttons[row][col]['text'] = "O"
        turn = not turn
        check_winner()
```

- Checks if the button is empty.
- Places “X” or “O” depending on the player turn.
- After every move, the program calls `check_winner()` to verify if the game is over.

6. Checking the Winner

```
def check_winner():
    # Check rows
    for i in range(3):
        if buttons[i][0]['text'] == buttons[i][1]['text'] ==
buttons[i][2]['text'] != " ":
            messagebox.showinfo("Tic Tac Toe", f"Player
{buttons[i][0]['text']} wins!")
            reset()
    return
```

- The function checks all possible winning conditions:
 - Rows
 - Columns
 - Diagonals
- If a winning condition is satisfied, the game declares the winner.
- If no winner and all cells are filled, the game declares a draw.

7. Resetting the Game

```
def reset():
    for i in range(3):
        for j in range(3):
            buttons[i][j]['text'] = " "
```

- This function clears the board to start a new game after completion.

8. Running the Application

```
root.mainloop()
```

- This line starts the Tkinter event loop and keeps the window active until the user closes it.

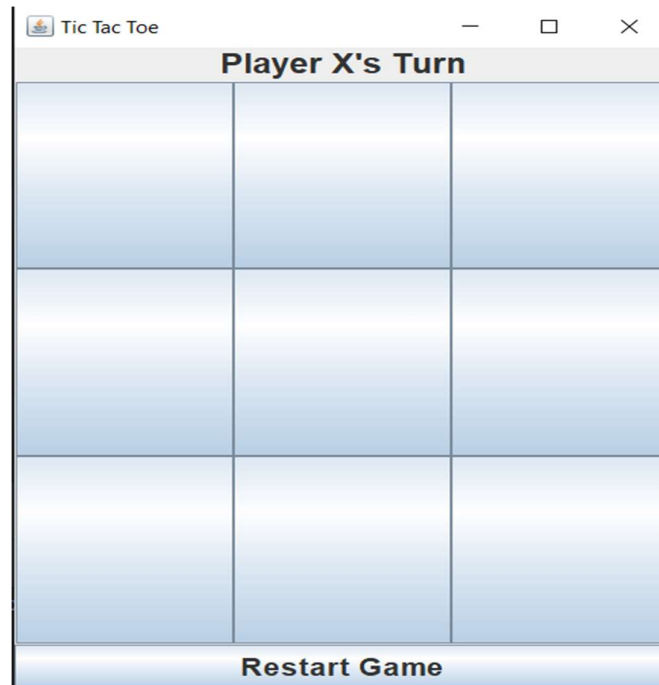
Output Screenshots:

The output section shows the working of the Tic Tac Toe game application. Screenshots are taken during different stages of the game to demonstrate its functionality.

1. Application Launch

When the program is executed, the Tic Tac Toe window appears with a **3×3 empty board**.

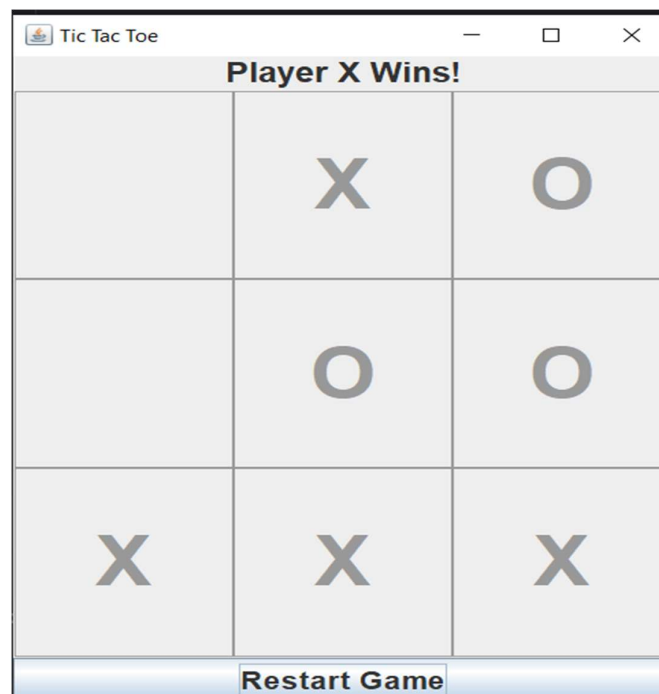
- Each cell is represented by a button.
- Initially, all cells are blank.
- Player X always gets the first turn.



2. Winning Condition

If a player successfully marks three consecutive cells in a row, column, or diagonal, the game declares that player as the winner.

- A message box pops up showing **“Player X Wins!”** or **“Player O Wins!”**.
- After that, the game board is reset automatically.



Testing and Validation

Testing is an essential phase of software development that ensures the correctness and reliability of the program. The Tic Tac Toe game was tested under multiple scenarios to verify its performance, accuracy, and error handling.

1. Functional Testing

This type of testing was performed to check whether all the functions of the game worked correctly.

- **Initial State Test:** Ensured that the game board is empty when the program starts.
- **Turn Alternation Test:** Verified that turns automatically switch between Player X and Player O.
- **Winning Condition Test:** Checked that the program correctly detects a win when a player marks three consecutive cells in a row, column, or diagonal.
- **Draw Condition Test:** Confirmed that when the board is full without a winner, the program displays a "Draw" message.
- **Reset Function Test:** Verified that the game resets correctly after a win or draw.

2. Boundary Testing

Boundary testing was used to test the limits of the program.

- Checked that no cell can be clicked twice.
- Ensured that the program does not allow invalid moves.
- Verified that after declaring a winner, no further moves can be made until the board is reset.

3. User Interface (UI) Testing

The graphical interface was tested for usability and correctness.

- All buttons were visible and aligned in a 3×3 grid.
- Messages such as "**Player X Wins**", "**Player O Wins**", and "**Game Draw**" were displayed correctly.
- Reset functionality worked smoothly without requiring a program restart.

4. Validation of Results

The game was validated by manually playing multiple rounds and comparing the output with expected results.

- Test cases matched the expected outcomes.
- The game successfully handled both winning and draw situations.
- No unexpected crashes or bugs occurred during validation.

4. Test Case Summary Table

Test Case ID	Description	Input/Action	Expected Output	Result
TC-01	Launch the game	Run the program	Empty 3×3 board displays	Pass
TC-02	Player X first move	Click any empty cell	Cell marked with "X"	Pass
TC-03	Turn alternation	Player O next move	Cell marked with "O"	Pass
TC-04	Winning condition (row)	X in 3 consecutive row	Message: "Player X Wins"	Pass
TC-05	Draw condition	Fill board no winner	Message: "Game Draw"	Pass
TC-06	Reset function	End game → reset board	Board cleared	Pass

Limitations of the Project:

Although the Tic Tac Toe game fulfills its objectives and provides a good demonstration of logic and GUI programming, like any software project, it has certain limitations. These limitations highlight the areas where the project can be improved in the future.

1. Limited Game Size

- The game is restricted to a **3×3 grid**.
- It cannot support larger board sizes such as 4×4 or 5×5, which could add variety and complexity.

2. Two-Player Restriction

- The current version only supports **two human players**.
- There is no option to play against a computer (AI opponent).

3. Lack of Difficulty Levels

- The game does not provide **easy, medium, or hard modes**.

- All games are played at the same difficulty level, which may reduce long-term engagement.

4. No Score Tracking

- The program does not keep track of the number of wins, losses, or draws over multiple rounds.
- Each game is independent and resets after a round ends.

5. Limited User Interface Features

- The graphical interface is simple and functional but lacks advanced design elements such as animations, sounds, or colorful themes.
- The lack of customization reduces user engagement.

6. No Online or Multiplayer Support

- The game can only be played locally on one computer.
- There is no feature for online play or multiplayer gaming over a network.

7. Limited Error Handling

- Although the program prevents invalid moves, it lacks advanced error handling for unexpected user actions.
- For example, closing the window abruptly may stop the game without a proper message.

Future Enhancements:

The Tic Tac Toe project can be further improved and extended in the future to make it more interactive, intelligent, and user-friendly. Below are some suggested enhancements:

1. Artificial Intelligence (AI) Opponent

- Adding an **AI player** would allow a single user to play against the computer.
- Algorithms like **Minimax** can be implemented to make the AI challenging.
- Difficulty levels (Easy, Medium, Hard) can be introduced for better gameplay experience.

2. Expanded Game Board

- Support for larger boards such as **4×4, 5×5, or custom sizes** can make the game more exciting and complex.
- This would allow players to enjoy more challenging variations of Tic Tac Toe.

3. Score Tracking System

- A feature to maintain scores (wins, losses, and draws) across multiple games.
- A scoreboard can be displayed at the end of each round and updated continuously.

4. Attractive User Interface

- Improving the GUI with **animations, sound effects, and themes**.
- Allowing players to customize colors, background images, or symbols (e.g., using emojis instead of “X” and “O”).

5. Online Multiplayer Mode

- Players should be able to connect and play with others over the internet or a local network.
- Chat features can be added to make the game more interactive.

6. Mobile Application Development

- Developing the same game as a **mobile app** for Android or iOS.
- A mobile-friendly interface would make the project more practical and accessible to a wide audience.

7. Save and Resume Feature

- Adding functionality to **save game progress** and resume it later.
- This is especially useful in multiplayer or longer versions of the game.

8. Advanced Error Handling

- More robust exception handling for unexpected events such as application crashes, sudden exit, or invalid inputs.
- This will make the program more reliable.

Conclusion of the Project:

The Tic Tac Toe Game project has been a valuable learning experience and a practical implementation of programming concepts. Through the development of this project, several important skills and ideas were explored, ranging from basic programming logic to user interface design.

Key Learnings

1. **Programming Logic:**

- The project reinforced the importance of conditional statements, loops, and functions in solving real-world problems.
- Implementation of game rules helped in understanding decision-making constructs and flow control in programming.

2. **Graphical User Interface (GUI):**

- By designing a GUI (if implemented), the project provided experience in event handling, layout design, and making programs interactive.
- This demonstrated how simple games can be turned into user-friendly applications.

3. **Problem-Solving Skills:**

- The process of detecting winning conditions, handling draw situations, and managing player turns helped in developing structured problem-solving abilities.

4. **Software Development Process:**

- From planning, designing, and coding to testing and debugging, the project gave exposure to the **complete software development cycle**.

Overall Contribution

- This project is not just a simple game but also a **foundation for future projects** involving artificial intelligence, networking, and mobile app development.
- It highlights how even a small application can showcase key programming principles and creativity.

References:

While preparing and implementing the Tic Tac Toe Game project, various resources were referred to for gaining knowledge, clarity, and guidance. The following references provided support during different stages of development:

Books

1. Herbert Schildt – *Java: The Complete Reference* (McGraw Hill)
2. E. Balagurusamy – *Programming in ANSI C* (McGraw Hill)
3. Robert Lafore – *Object-Oriented Programming in C++* (Sams Publishing)
4. Yashavant Kanetkar – *Let Us C* (BPB Publications)

Websites & Online Resources

1. w3schools.com – For basic programming concepts and syntax references.
2. geeksforgeeks.org – For algorithms, logic implementation, and problem-solving techniques.
3. tutorialspoint.com – For learning object-oriented concepts and Java basics.
4. [stack overflow](https://stackoverflow.com) – For debugging issues and understanding common coding errors.

Software & Tools

1. **Java Development Kit (JDK)** – For compiling and running Java code.
2. **IDE (IntelliJ IDEA / Eclipse / NetBeans)** – For efficient coding and debugging.
3. **MS Word** – For preparing the project report.
4. **Draw.io / Lucidchart** – For creating flowcharts and diagrams.

Acknowledged Resources

The project also benefited from various **online tutorials, YouTube programming channels**, and **college lecture notes** which helped in improving understanding of logic and implementation.

Appendix

The appendix provides additional supporting material related to the Tic Tac Toe Game project. This section contains **code snippets, screenshots placeholders, and extra notes** that complement the main report.

A. Sample Code Snippets

1. Main Game Logic (Java Example)

```
// Checking Winning Condition
if (board[0][0] == player && board[0][1] == player && board[0][2] ==
player) {
    return true; // Row 1 win
```

```

    }
    if (board[1][0] == player && board[1][1] == player && board[1][2] ==
player) {
        return true; // Row 2 win
    }
    if (board[2][0] == player && board[2][1] == player && board[2][2] ==
player) {
        return true; // Row 3 win
    }

    // Diagonal check
    if (board[0][0] == player && board[1][1] == player && board[2][2] ==
player) {
        return true;
    }
}

2. Displaying the Board
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(board[i][j] + " ");
    }
    System.out.println();
}

```

Future Scope of the Project

The Tic Tac Toe Game project, though simple in design, has significant potential for further development. The following points outline the **future scope** of this project:

1. Artificial Intelligence (AI) Integration

- The current version allows two players to play manually.
- In the future, a computer-controlled opponent can be developed using algorithms like **Minimax**, which would make the game more challenging.

2. Graphical User Interface (GUI) Enhancements

- Adding attractive visuals, animations, and sound effects would make the game more engaging.
- A responsive design can also be built for mobile devices.

3. Online Multiplayer Mode

- The game can be enhanced to allow players to connect online and compete with each other.
- This feature can be implemented using **network programming** or integration with cloud services.

4. Difficulty Levels

- Introducing easy, medium, and hard levels with different strategies for the AI opponent.
- This would make the game enjoyable for both beginners and advanced players.

5. Score Tracking and Leaderboards

- A database system can be integrated to keep track of scores and display player rankings.
- This adds competitiveness and replay value.

6. Educational Purpose

- The project can be used as a learning tool for beginners in programming to understand arrays, loops, conditionals, and algorithms.

Conclusion

The Tic Tac Toe Game project has been a valuable learning experience that demonstrates the application of basic programming concepts in a structured way. It successfully implements the logic of the game using fundamental concepts such as loops, conditionals, arrays, and functions.

Through this project, I gained hands-on knowledge of designing, coding, debugging, and testing a small-scale application. The game, though simple, reflects important problem-solving skills and logical thinking abilities.

This project also highlights how a small idea can be expanded into a larger system in the future, such as adding artificial intelligence, online multiplayer functionality, and advanced user interfaces.

In conclusion, the Tic Tac Toe Game is not only a fun and interactive application but also an effective project for strengthening programming knowledge and building a foundation for developing more complex software projects in the future.